MICROPROCESSOR & MICROCONTROLLER



WATCHDOG TIMER

(WDG)

Functionality, Working Principle & Real-World Use Cases

PRESENTED BY

Md Habibur Rahman	2210376108
Soumithra Debnath Shanto	2210976136
Md Habibur Rahman	2210976107

AGENDA

What We Will Cover

1. Introduction

What a watchdog timer is, and why embedded systems need one

2. Types of Watchdog

Independent (IWDG) vs Window (WWDG) watchdogs

3. How It Works

The core countdown-and-refresh mechanism, register level

4. STM32 Implementation

IWDG & WWDG architecture on the STM32F103C8T6

5. Use Cases

Automotive, industrial, IoT, medical & aerospace applications

6. Pros, Cons & Summary

Advantages, limitations and best practices

INTRODUCTION

What Is a Watchdog Timer?

FULL FORM

WDG = Watchdog Timer

A hardware timing circuit built into the microcontroller that monitors the health of running software.

Definition

A Watchdog Timer (WDG) is a free-running hardware counter that automatically resets the microcontroller if the application software fails to periodically restart ("refresh" or "kick") it in time — for example, when the CPU is stuck in an infinite loop, a deadlock, or a corrupted execution path.

Why It Is Needed

Embedded systems often run unattended for months or years — a car ECU, a remote sensor, a factory controller. They cannot wait for a human to notice a freeze and press reset. The watchdog gives the system the ability to detect its own malfunction and recover automatically, without any manual intervention.



The Guard-Dog Analogy

Think of the watchdog as a guard dog that must be "petted" (refreshed) regularly. As long as you keep petting it on schedule, it stays calm. The moment you stop — because you are asleep, stuck, or unable to respond — it "barks": it resets the system.

"Pet on time → system runs.
Miss the pet → system resets."

CLASSIFICATION

Two Types of Watchdog Timers



IWDG

Independent Watchdog

- Runs on its own separate low-speed clock (LSI, ~40 kHz on STM32)
- Fully independent of the main system clock
- Keeps working even if the main oscillator fails or is misconfigured
- Simple: must be refreshed before a single timeout deadline
- Best for detecting total software lock-ups / hangs



WWDG

Window Watchdog

- Runs on the main peripheral clock (APB1) with a prescaler
- Must be refreshed inside a specific time "window"
- Refreshing too early OR too late both trigger a reset
- Catches both slow tasks and abnormally fast / skipped tasks
- Best for detecting timing & sequencing faults

HOW IT WORKS

The Core Countdown-and-Refresh Mechanism



A timeout value is loaded into the watchdog's down-counter register at startup.

The counter decreases continuously, driven by its own clock — independent of the CPU's main program.

The application must periodically write a specific "refresh" value to reload the counter before it hits zero.

If the refresh never arrives in time — because code is hung — the counter reaches zero.

A reset signal is generated automatically, restarting the microcontroller from its boot vector.

Key Idea: The watchdog runs independently of the main program. If the CPU is "alive" but stuck in a bad loop, only an independent timer can detect it — the frozen code cannot detect its own freeze.

STM32 IMPLEMENTATION

Independent Watchdog (IWDG) — Register View



Key Registers

`IWDG_KR`	Key register — write 0xCCCC to start, 0xAAAA to refresh, 0x5555 to unlock config
`IWDG_PR`	Prescaler register — sets the clock divider (÷4 to ÷256)
`IWDG_RLR`	Reload register (12-bit) — sets the timeout / down-count start value
`IWDG_SR`	Status register — flags when PR/RLR updates are still in progress

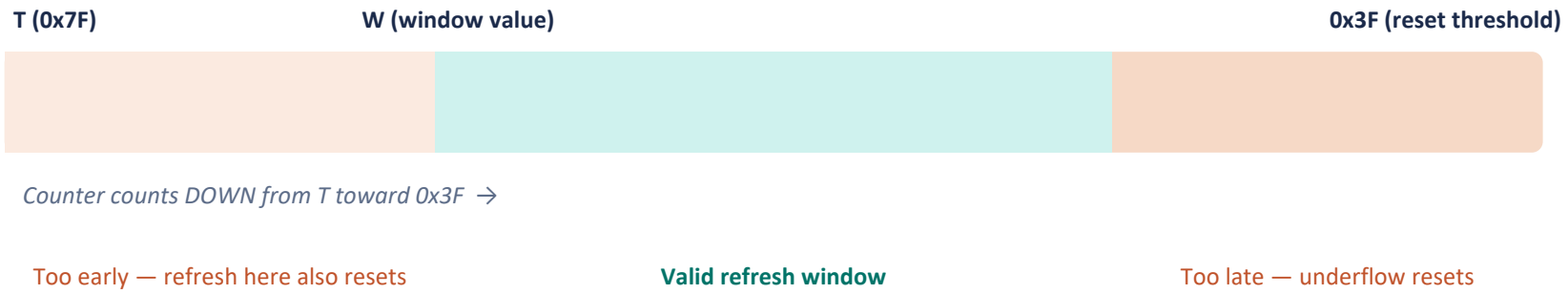
EXAMPLE — refresh call inside main loop

```
IWDG->KR = 0x5555;      // unlock PR & RLR
IWDG->PR  = 0x04;        // set prescaler
IWDG->RLR = 0x0FFF;     // set reload / timeout
IWDG->KR = 0xCCCC;      // start the watchdog
while (1) {
   run_application_tasks();
   IWDG->KR = 0xAAAA;   // refresh (kick)
}
```

STM32 IMPLEMENTATION

Window Watchdog (WWDG)

Unlike the IWDG, the WWDG defines a valid **window** of time to refresh in — bounded by an upper limit (T) and a lower limit (W). Refreshing outside this window, in either direction, triggers a reset.



Why a window at all?

A single-timeout watchdog (like IWDG) only catches code that is too slow. The WWDG additionally catches code that runs abnormally fast — e.g. a task that skips steps due to memory corruption or a jump to the wrong address. This makes it well suited for safety-critical timing supervision, such as verifying a control loop executes at a precise, expected rate.

Key Registers

`WWDG_CR`

7-bit down-counter T[6:0] + watchdog enable bit

`WWDG_CFR`

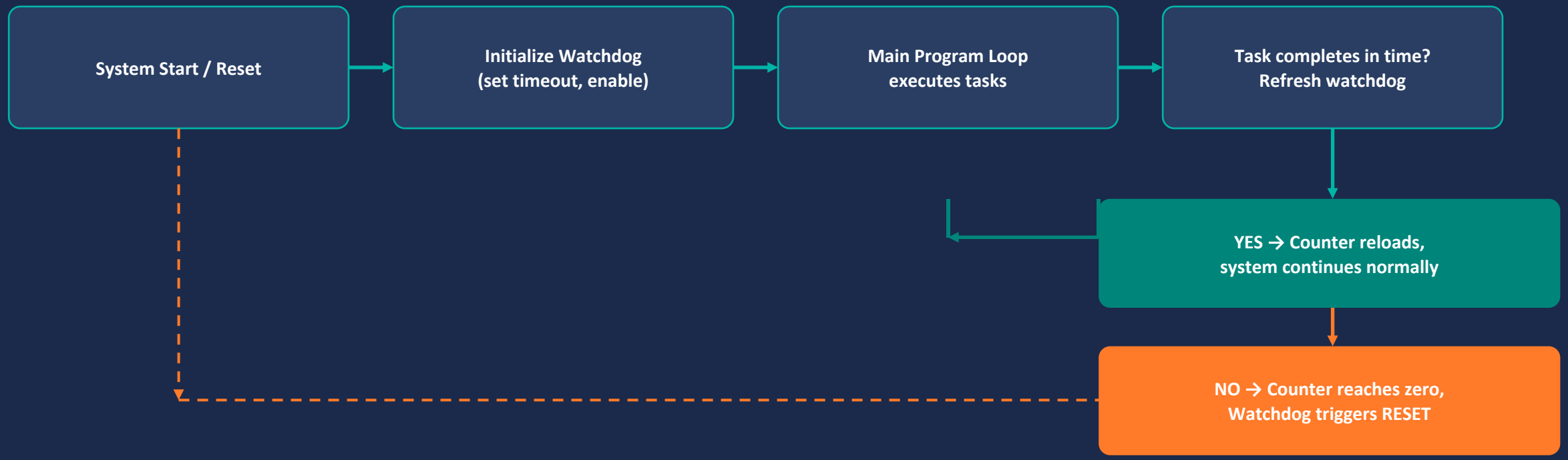
Window value W[6:0] + clock prescaler (WDGTB)

`WWDG_SR`

Early-wakeup interrupt flag (EWIF)

PROCESS FLOW

Watchdog Operation — End to End



Dashed line: a reset restarts execution from the boot vector, re-entering the flow at "System Start."

APPLICATIONS

Where Watchdog Timers Are Used



Automotive ECUs

Engine, braking & airbag control units use watchdogs to guarantee fail-safe recovery in safety-critical systems.



Industrial Automation

PLCs and factory controllers run unattended for years; a watchdog keeps production lines from silently freezing.



IoT & Remote Sensors

Field-deployed nodes cannot be manually rebooted — the watchdog is the only recovery mechanism available.



Medical Devices

Patient monitors and infusion pumps require guaranteed responsiveness; a hang must self-correct immediately.



Consumer Electronics

Routers, smart appliances & set-top boxes silently recover from firmware glitches without user action.



Aerospace Systems

Satellites and autonomous craft are physically unreachable — onboard watchdogs are the last line of defense.

EVALUATION

Advantages & Limitations

Advantages

- Automatic recovery from software hangs — no human intervention needed
- Significantly improves long-term system reliability & uptime
- Catches faults regular software checks cannot (deadlocks, infinite loops)
- Very low hardware cost and negligible power overhead
- Simple to implement — a handful of register writes

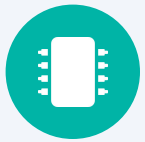
Limitations

- Resets the symptom, not the root cause — the underlying bug remains
- Careless refresh placement (e.g. inside an ISR) can mask real hangs
- Cannot detect faults where code stays "alive" but produces wrong output
- Timeout value must be tuned carefully — too short causes false resets
- Frequent unexplained resets can indicate a deeper design problem

SUMMARY

Conclusion

The Watchdog Timer is a small but essential piece of hardware: a self-monitoring safety net that keeps embedded systems reliable, autonomous, and recoverable — without which unattended electronics could freeze permanently with no way to recover.



Two flavors

IWDG for independent fault detection; WWDG for precise timing-window supervision.



Refresh discipline

Correct placement of the "kick" call is critical — it must only run when the system is truly healthy.



Safety-critical role

Standard practice across automotive, industrial, medical, IoT & aerospace embedded design.



Thank You

Questions & Discussion Welcome